

14

Lecture 14

**Classic Synchronization Problems I:
Producer-Consumer**

by Brijesh Shukla
C07: Operating Systems

Join the lecture online on your dashboard

Today's Agenda

- Review of Semaphores
- Producer-Consumer Problem Statement
- Synchronization Requirements
- Semaphore-Based Solution
- Code Walkthrough and Analysis
- Variations and Extensions



Introduction to Synchronization

What is Process Synchronization?

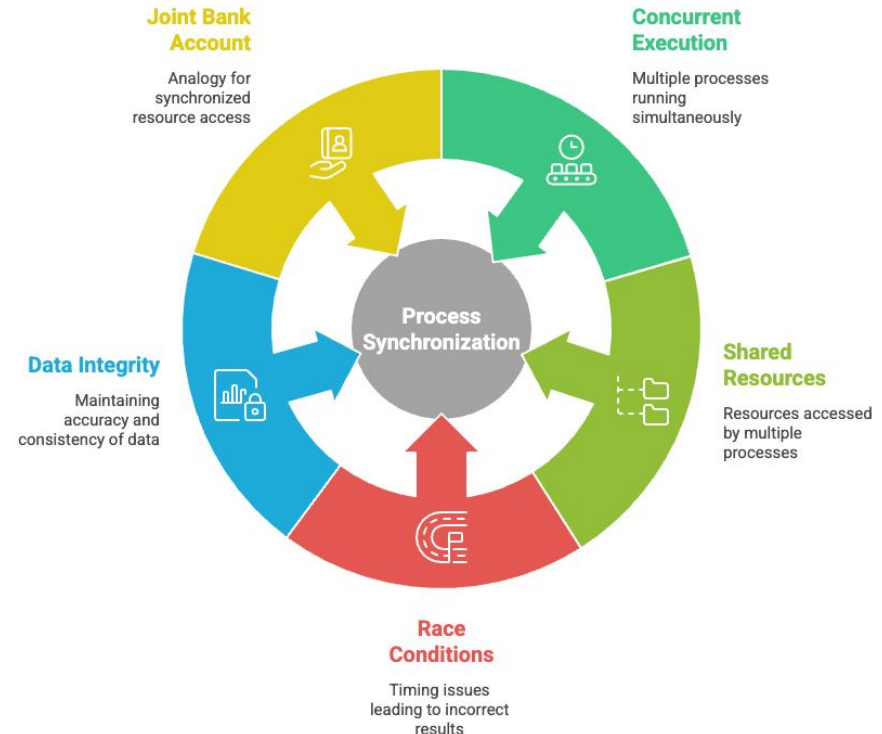
- Mechanism to **safely coordinate multiple processes/threads**
- Ensures **correct access to shared resources**
- Prevents **race conditions** and **data inconsistency**

Why Do We Need Synchronization?

-  **Concurrent execution** of processes
-  **Shared resources** (memory, files, data structures)
-  **Race conditions** due to timing issues
-  **Data integrity** must be preserved

Real-World Analogy

-  **Joint bank account**
- Two people withdraw money **at the same time**
-  Without synchronization → **incorrect balance**
-  Synchronization ensures **one update at a time**



Review of Semaphores – Basics

What is a Semaphore?

- A **synchronization primitive**
- Uses an **integer variable**
- Accessed only via **atomic operations**:
 -  `wait()` (P / down)
 -  `signal()` (V / up)

Types of Semaphores

	Binary Semaphore (Mutex)	Counting Semaphore
Value	010 101 0 or 1	 0 to n
Purpose	Mutual exclusion	Multiple identical resources
Analogy	Lock	Resource counter
Usage	One process at a time	Controls resource availability

Semaphore Operations

Atomic Operations:

wait(S) or P(S):

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

signal(S) or V(S):

```
signal(S) {  
    S++;  
}
```

★ Key Properties

-  **Atomicity** – executes as one indivisible step
-  **No interruption** during operation
-  **Mutual exclusion** for shared resources

⚠ Important Note

-  **Busy waiting** wastes CPU
-  Real systems use **blocking (sleep & wakeup)**

Producer-Consumer Problem – Introduction

What is the Producer-Consumer Problem?

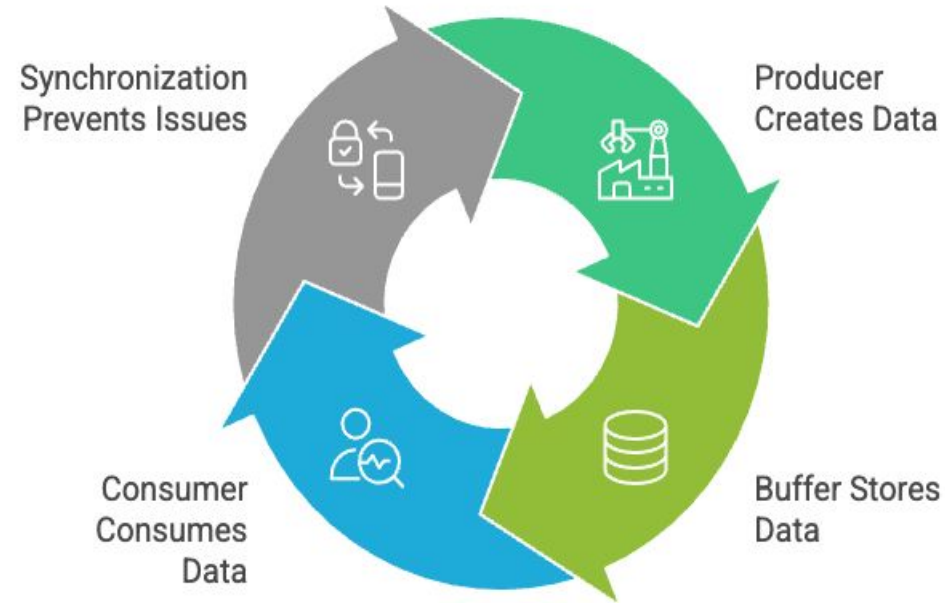
- Also called the **Bounded Buffer Problem**
- A **classic synchronization problem**
- Involves two types of processes:
 -  **Producer** → creates data, puts into buffer
 -  **Consumer** → removes data, processes it

Real-World Examples

-  **Print Spooler**: Apps produce jobs → Printer consumes
-  **Web Server**: Server produces pages → Browser consumes
-  **Assembly Line**: Production → Packaging
-  **Message Queue**: Sender → Receiver

Key Idea

- Shared buffer requires **synchronization**
- Prevents: Buffer overflow. Buffer underflow. Race conditions



Problem Statement

✖ Bounded Buffer Scenario

- 📦 **Buffer**: Fixed-size array with **N slots**
- 🟢 **Producers**: Generate data items
- 🔵 **Consumers**: Consume data items

🔄 Operations

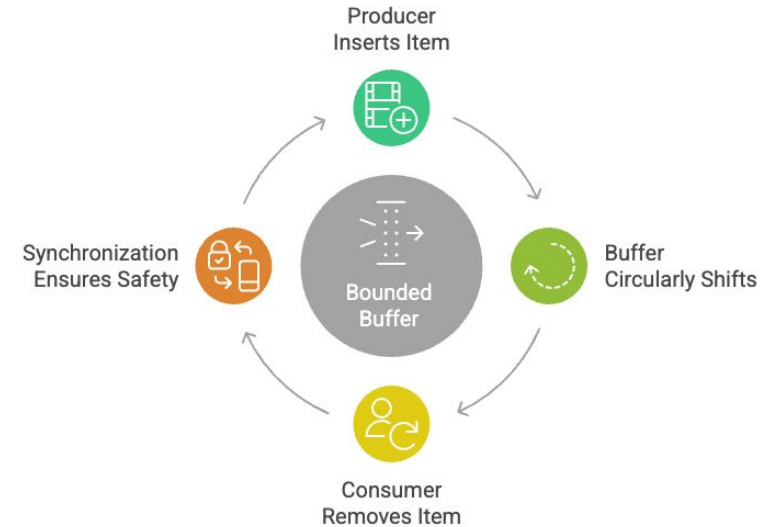
- ➕ Producer inserts item at **in**
- ➖ Consumer removes item from **out**
- 🔄 Buffer works as a **circular (ring) buffer**

👁 Conceptual View

Producer → [I1] [I2] [I3] [_] [_] → **Consumer**

🎯 Key Points

- Buffer size is **limited**
- Requires **synchronization**
- Prevents: ❌ Overflow (buffer full). ❌ Underflow (buffer empty)



Synchronization Requirements

1 Mutual Exclusion

-  Only **one process** accesses buffer at a time
- Prevents **race conditions**
- Protects buffer state (**in**, **out**, data)

2 Producer Must Wait

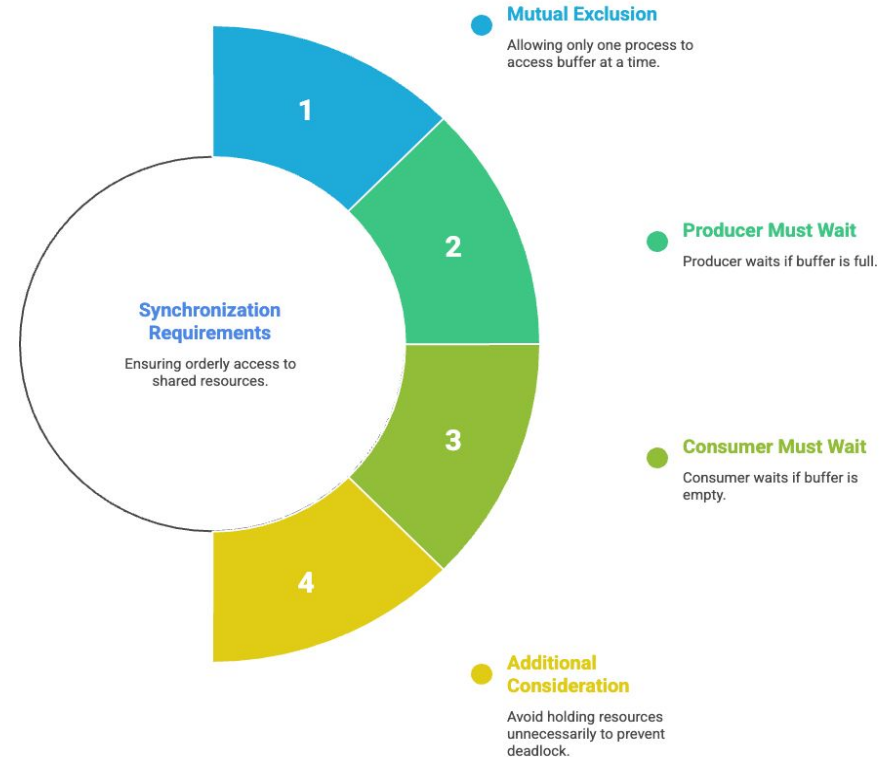
-  Buffer is **FULL**
- No empty slots available
- Prevents **buffer overflow**

3 Consumer Must Wait

-  Buffer is **EMPTY**
- No items to consume
- Prevents **buffer underflow**

Additional Consideration

-  Avoid holding resources unnecessarily
- Prevents **deadlock**



Identifying Shared Resources

What Needs to be Synchronized?

Shared Variables

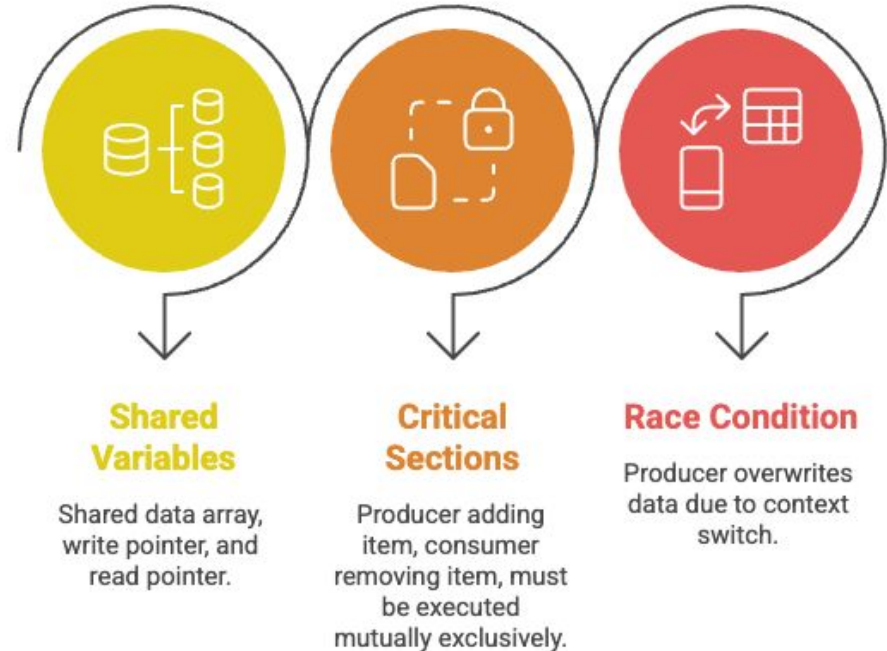
- `buffer[N]` → Shared data array
- `in` → Write pointer (producer index)
- `out` → Read pointer (consumer index)

Critical Sections

-  Producer **adding item** to buffer
-  Consumer **removing item** from buffer
- Must be executed **mutually exclusively**

! Race Condition (Without Sync)

- Producer reads `in = 5`
-  Context switch occurs
- Another producer writes at `in = 5`
-  First producer resumes → **overwrites data**



Semaphore-Based Solution – Overview

Semaphores Required

- 1 **mutex** (Binary Semaphore)
- 2 **empty** (Counting Semaphore)
- 3 **full** (Counting Semaphore)

Characteristic	Mutex	Empty	Full
Initial Value	1	N	0
Purpose	Ensures mutual exclusion	Counts empty slots	Counts filled slots
Wait Condition	Protects critical section	Producer waits if buffer is full	Consumer waits if buffer is empty

Key Insight

- **Counting semaphores** → track resource availability (**empty**, **full**)
- **Binary semaphore** → ensures safe access (**mutex**)
- Together → **correct & safe synchronization**

Producer Process – Code

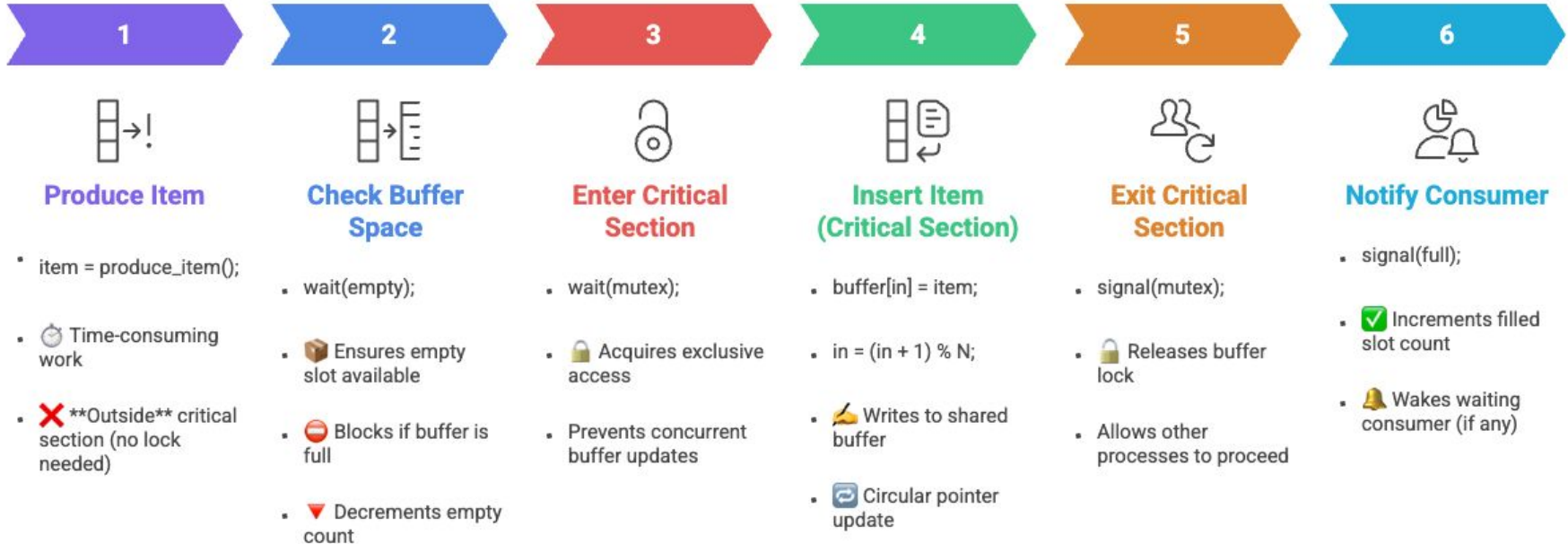
Producer Algorithm

```
void producer() {  
    int item;  
  
    while (TRUE) {  
        item = produce_item();    // Produce item (outside CS)  
        wait(empty);              // Wait for empty slot  
        wait(mutex);              // Enter critical section  
        buffer[in] = item;        // Insert item  
        in = (in + 1) % N;        // Circular buffer update  
        signal(mutex);           // Exit critical section  
        signal(full);             // One more full slot  
    }  
}
```

Key Points

-  `wait(empty)` → ensures buffer is **not full**
-  `wait(mutex)` → **mutual exclusion**
-  Circular buffer using `(in + 1) % N`
-  `signal(full)` → notifies consumer

Producer Code – Step-by-Step Analysis



Consumer Process – Code

Consumer Algorithm

```
void consumer() {  
    int item;  
  
    while (TRUE) {  
        wait(full);           // Wait for available item  
        wait(mutex);         // Enter critical section  
  
        item = buffer[out];   // Remove item  
        out = (out + 1) % N;   // Circular buffer update  
  
        signal(mutex);       // Exit critical section  
        signal(empty);       // One more empty slot  
  
        consume_item(item);   // Consume item (outside CS)  
    }  
}
```

Key Points

-  `wait(full)` → ensures buffer is **not empty**
-  `wait(mutex)` → **exclusive buffer access**
-  Circular buffer using `(out + 1) % N`
-  `signal(empty)` → notifies producer

Consumer Code - Step-by-Step Analysis



Why This Solution Works

1. Mutual Exclusion

- **mutex** ensures **one process at a time**
- Prevents **simultaneous buffer access**

2. Bounded Buffer

- **empty** → blocks producer when buffer is **full**
- **full** → blocks consumer when buffer is **empty**
- Prevents **overflow & underflow**

3. No Deadlock

- Proper **resource ordering**
- Semaphores correctly track availability
- No circular wait

4. Progress

- Processes run when resources are available
- **signal()** wakes waiting processes
- No unnecessary blocking

5. Bounded Waiting

- Semaphore queues follow **FIFO**
- Ensures **fairness**
- No starvation



Mutual Exclusion

Ensures only one process accesses the buffer at a time.



Bounded Buffer

Prevents buffer overflow and underflow by blocking processes.



No Deadlock

Avoids circular resource waits through proper ordering.



Progress

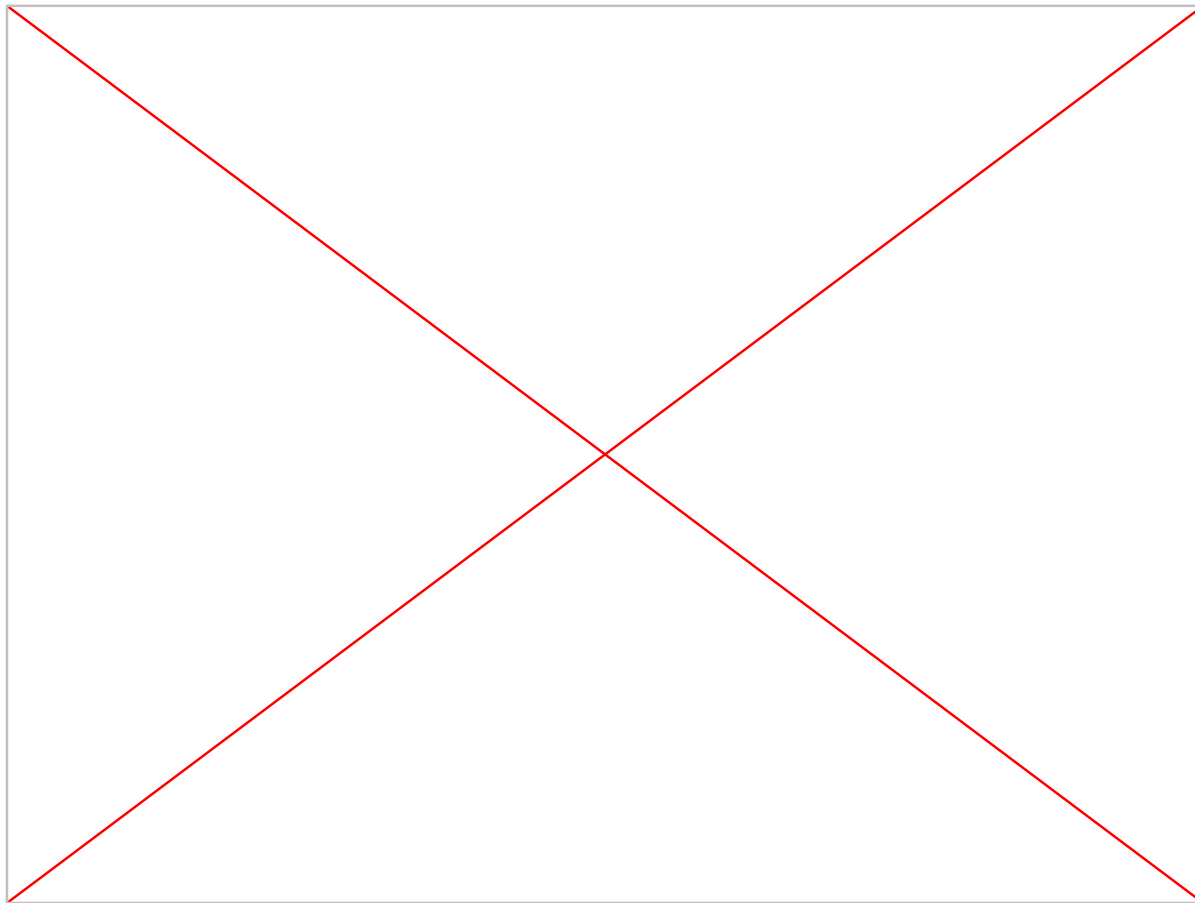
Allows processes to run when resources are available.



Bounded Waiting

Guarantees fairness by processing requests in FIFO order.

Meme Break



Common Mistake – Wrong Semaphore Order

✗ Incorrect Producer Code

```
void producer() {  
    item = produce_item();  
    wait(mutex);  
    wait(empty);  
    buffer[in] = item;  
    in = (in + 1) % N;  
    signal(mutex);  
    signal(full);  
}
```

💥 Why This Causes Deadlock

- Producer **locks mutex**
- Buffer is **full** → blocks on `wait(empty)`
- Consumer **cannot acquire mutex**
- Consumer cannot execute `signal(empty)`
- ✗ **Deadlock**: both wait forever

🚫 Root Cause

- Holding **mutex while waiting** for resources
- Violates **proper semaphore ordering**

✓ **Correct Rule: Wait for resources first. Then acquire mutex. Release mutex before signaling**

Variations – Multiple Producers/Consumers

Same Solution Applies: Works for **1 or many producers**. Works for **1 or many consumers**. No code changes required.

● Multiple Producers

```
void producer_i() {  
    while (TRUE) {  
        item = produce_item();  
        wait(empty);  
        wait(mutex);  
        buffer[in] = item;  
        in = (in + 1) % N;  
        signal(mutex);  
        signal(full);  
    }  
}
```

● Multiple Consumers

```
void consumer_j() {  
    while (TRUE) {  
        wait(full);  
        wait(mutex);  
        item = buffer[out];  
        out = (out + 1) % N;  
        signal(mutex);  
        signal(empty);  
        consume_item(item);  
    }  
}
```

✓ Why It Works

-  Semaphores **manage waiting queues**
-  Multiple processes can block on same semaphore
-  FIFO ordering ensures **fairness**
-  No race conditions or deadlock

Variation – Unbounded Buffer

🧠 **When Buffer Has No Size Limit:** Buffer has **infinite capacity**. No overflow possible. Synchronization becomes simpler.

🔧 Semaphores Used

```
semaphore mutex = 1;    // Mutual exclusion
semaphore full  = 0;    // Items available
// ❌ No 'empty' semaphore
```

🟢 Producer

```
void producer() {
    while (TRUE) {
        item = produce_item();
        wait(mutex);
        append_to_buffer(item);
        signal(mutex);
        signal(full);
    }
}
```

🟡 Consumer

```
void consumer() {
    while (TRUE) {
        wait(full);
        wait(mutex);
        item = remove_from_buffer();
        signal(mutex);
        consume_item(item);
    }
}
```

🔑 Key Differences (vs Bounded Buffer)

- ❌ No **empty** semaphore
- 🟢 Producer **never blocks** (infinite space)
- 🟡 Consumer blocks **only when buffer is empty**
- 🔒 **mutex** still required for safety

Variation – Single Producer, Single Consumer

🧠 **Scenario: Only 1 Producer. Only 1 Consumer.** Shared buffer with size N.

🔧 Simplified Semaphores

- semaphore **empty** = N; // Empty slots
- semaphore **full** = 0; // Filled slots
- // ❌ No mutex needed

🟢 Producer

```
void producer() {  
    while (TRUE) {  
        item = produce_item();  
        wait(empty);  
        buffer[in] = item;  
        in = (in + 1) % N;  
        signal(full);  
    }  
}
```

🟢 Consumer

```
void consumer() {  
    while (TRUE) {  
        wait(full);  
        item = buffer[out];  
        out = (out + 1) % N;  
        signal(empty);  
        consume_item(item);  
    }  
}
```

✅ Why Mutex Is NOT Needed

- 🟢 in updated **only by producer**
- 🟢 out updated **only by consumer**
- ❌ No shared-variable conflict
- ❌ No race condition

Performance Considerations

Optimizing the Solution

① Minimize Critical Section

-  Keep **only buffer access** inside lock
-  Move `produce_item()` / `consume_item()` **outside**

② Avoid Busy Waiting

-  Use **blocking semaphores** (sleep / wakeup)
-  Avoid spinlocks for long waits

③ Buffer Size Selection

-  **Large buffer** → higher throughput, more memory
-  **Small buffer** → less memory, more blocking
-  Choose size based on **producer-consumer rate**

④ Lock Granularity

-  **Coarse-grained lock** → simple, less concurrency
-  **Fine-grained locks** → more concurrency, complex design

Key Performance Metrics

-  **Throughput** → items processed per second
-  **Latency** → time from produce to consume
-  **CPU Utilization** → useful work vs waiting



Key Performance Metrics

Measures of solution performance.



Throughput

Items processed per second.



Latency

Time from produce to consume.



CPU Utilization

Useful work vs waiting.

Real-World Applications

Application Area	Examples
Operating Systems	I/O buffering, Pipes, Keyboard buffer
Multithreaded Applications	Thread pools, Event handling, Logging systems
Distributed Systems	Message queues, Load balancers, Stream processing pipelines
Embedded Systems	Sensor data processing, Audio/video streaming pipelines

Key Takeaway

- Producer–Consumer is a **core design pattern**
- Used from **kernels to cloud systems**

Common Pitfalls and Debugging

⚠ Frequent Mistakes

❶ Wrong Semaphore Order

```
wait(mutex);    // ❌ Wrong  
wait(empty);    // Deadlock risk  
Blocks while holding mutex
```

❷ Missing signal()

```
wait(mutex);  
buffer[in] = item;  
// ❌ signal(mutex) missing → permanent lock
```

❸ Mismatched wait() / signal()

```
wait(empty);  
// ...  
signal(full); // ❌ Wrong pairing  
Breaks resource accounting
```

🔍 Debugging Techniques

- 📄 Trace execution (log semaphore values)
- 🧮 Check invariants: $\text{empty} + \text{full} = N$
- 🔁 Deadlock detection (circular wait)



Practice Problems

 Given (Buggy?) Producer Code

```
void producer() {  
    int item;  
    while (1) {  
        item = produce_item();  
        wait(mutex);  
        wait(empty);  
        buffer[in] = item;  
        in = (in + 1) % 10;  
        signal(mutex);  
        signal(full);  
    }  
}
```

Question

- What is the **bug**?
- Why is it a **problem**?
- How to **fix** it?

✗ Bug Identified

- **Wrong semaphore order**
- `wait(mutex)` is called **before** `wait(empty)`

💥 Why This Is a Problem

- If buffer is **full** (`empty = 0`):
 - Producer blocks at `wait(empty)`
 - **While holding `mutex`**
- Consumer cannot acquire `mutex`
- ✗ **Deadlock occurs**

✓ Correct Fix (Swap Order)

```
wait(empty);    // ✓ Check resource first  
wait(mutex);    // ✓ Then enter critical section
```

Practice Problems

Task: Fill in the missing semaphore operations in the consumer code.

```
void consumer() {
    int item;
    while (1) {
        // Line 1: Wait for item availability
        -----
        // Line 2: Acquire buffer lock
        -----
        item = buffer[out];
        out = (out + 1) % 10;
        // Line 3: Release buffer lock
        -----
        // Line 4: Signal empty slot
        -----
        consume_item(item);
    }
}
```

✓ Expected Answer

```
void consumer() {
    int item;
    while (1) {
        wait(full);           // Line 1
        wait(mutex);          // Line 2
        item = buffer[out];
        out = (out + 1) % 10;
        signal(mutex);        // Line 3
        signal(empty);         // Line 4
        consume_item(item);
    }
}
```

🎯 Key Rule (Remember!)

- Check availability first (`wait(full)`)
- Then lock buffer (`wait(mutex)`)
- Unlock before signaling
- Consume outside the critical section

Key Takeaways

Core Semaphores

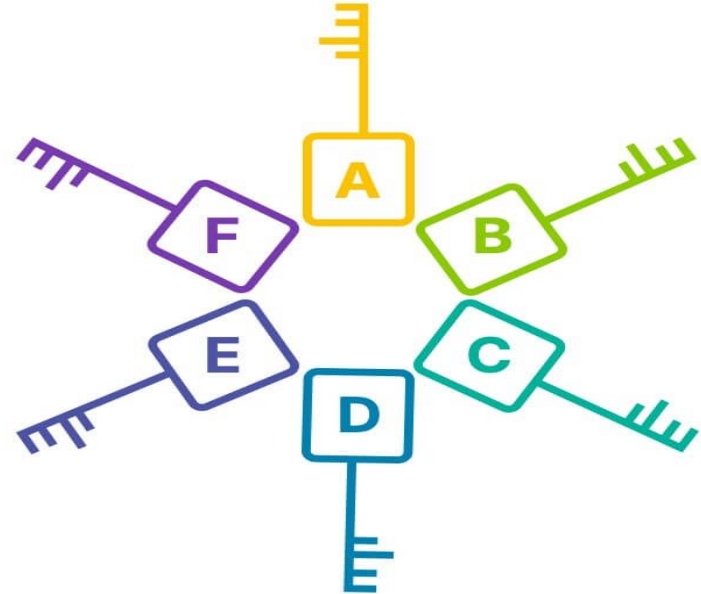
- **mutex** = 1 → Mutual exclusion
- **empty** = N → Empty buffer slots
- **full** = 0 → Filled buffer slots

Correct Semaphore Order

- **Producer:** **wait**(empty) → **wait**(mutex) → **signal**(mutex) → **signal**(full)
- **Consumer:** **wait**(full) → **wait**(mutex) → **signal**(mutex) → **signal**(empty)

Golden Rules

- ✓ Wait for **resources** before **mutex**
- ✓ Keep **critical sections** small
- ✓ Every **wait()** must have a matching **signal()**
- ✓ Always test with **multiple producers & consumers**



Thank You!

Thank you for your attention. Feel free to ask questions or revisit the topics covered.